OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses machine learning to provide intelligent, data-driven crop recommendations for farmers, agricultural researchers, and policymakers. The platform includes a Crop Recommendation module, which analyzes seven soil and environmental parameters — Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall — to predict the most suitable crop; a Model Comparison module, which trains and evaluates KNN, Logistic Regression, Decision Tree, and Random Forest to select the best performer; and an exploratory K-Means Clustering module that groups similar agricultural conditions.

Built with Flask and scikit-learn, OptiCrop processes user-submitted soil and climate values to deliver fast, reliable crop predictions through a simple web interface. The trained model and scaler are persisted with Pickle so predictions happen instantly without retraining. With input validation and a clean, farmer-friendly interface, OptiCrop empowers farmers, researchers, and policymakers to make evidence-based agricultural decisions with confidence.

Scenarios

Scenario 1 — Smart Crop Recommendation for Farmers: A farmer enters soil and environmental details such as N, P, K, temperature, humidity, pH, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

Scenario 2 — Crop Suitability and Environmental Assessment: A user wants to understand whether current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insight into crop compatibility, environmental suitability, and productivity potential.

Scenario 3 — Agricultural Research and Policy Planning: An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production, supporting data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture:

Description: OptiCrop uses a modular architecture to deliver fast, ML-driven crop recommendations. The frontend provides a responsive interface built with HTML and CSS, while the Flask-based backend handles input validation and prediction requests. It loads a pre-trained scikit-learn model (Random Forest, selected after comparing four algorithms) and a fitted StandardScaler from Pickle files to generate predictions with no external API dependency.

Note: OptiCrop does not use a traditional database — the training dataset is a static CSV file and the trained model/scaler are stored as Pickle files, so no ER diagram is required. (See the Entity Relationship Diagram document for the conceptual data model used for planning purposes.)

Pre-requisites:

- **Flask Framework Knowledge:** https://flask.palletsprojects.com/
- **Scikit-learn Familiarity:** https://scikit-learn.org/stable/
- **Pandas & NumPy Skills:** https://pandas.pydata.org/docs/ , https://numpy.org/doc/stable/
- **Python Programming Proficiency:** https://docs.python.org/3/
- **Version Control with Git:** https://git-scm.com/doc
- **Jupyter Notebook / Anaconda Setup:** https://www.anaconda.com/download
- **HTML & CSS Skills:** https://www.w3schools.com/html/ , https://www.w3schools.com/css/

Project Workflow:

Milestone 1: Data Collection and Model Selection

• **Activity 1.1:** Collect/generate the crop recommendation dataset (N, P, K, temperature, humidity, pH, rainfall, label).

• **Activity 1.2:** Research and select appropriate ML algorithms (KNN, Logistic Regression, Decision Tree, Random Forest, K-Means) for crop classification.

• **Activity 1.3:** Define the application architecture, detailing interactions between the frontend, Flask backend, and the trained ML model.

• **Activity 1.4:** Set up the development environment, installing Flask, scikit-learn, pandas, numpy, matplotlib, and seaborn.

Milestone 2: Data Preprocessing and EDA

• **Activity 2.1:** Perform univariate, bivariate, and multivariate analysis to understand feature distributions and relationships.

• **Activity 2.2:** Check for null values, detect and handle outliers (e.g., log-transform Potassium), extract seasonal crop groupings, and split data into train/test sets.

Milestone 3: Model Building

• **Activity 3.1:** Train KNN, Logistic Regression, Decision Tree, and Random Forest classifiers; explore K-Means clustering on scaled features.

• **Activity 3.2:** Evaluate all models using Accuracy, Precision, Recall, and F1-Score; select the best-performing model and save it (with its scaler) using Pickle.

Milestone 4: Flask Backend Development

• **Activity 4.1:** Write app.py, defining routes for Home, About, and Find Your Crop, and loading the trained model/scaler at startup.

• **Activity 4.2:** Implement the /findyourcrop POST handler — read form input, validate it, scale it, and return the predicted crop.

Milestone 5: Frontend Development

• **Activity 5.1:** Build the Home, About, and Find Your Crop HTML pages with a shared navigation bar.

• **Activity 5.2:** Style the interface with a single shared stylesheet (static/css/style.css) for a clean, responsive look.

Milestone 6: Deployment and Conclusion

• **Activity 6.1:** Run and verify the application locally using the Flask development server.

• **Activity 6.2:** Document results, limitations, and future scalability plans.

Milestone 3: Model Building — Implementation Detail

Milestone 3 focuses on training and comparing multiple algorithms in train_model.py, which mirrors the exploratory notebook used during EDA.

Activity 3.1 — Training and comparing models:

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42, max_depth=10),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    results[name] = {
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds, average="weighted"),
        "Recall": recall_score(y_test, preds, average="weighted"),
        "F1-Score": f1_score(y_test, preds, average="weighted"),
    }
```

Activity 3.2 — Saving the best model:

```
best_name = results_df.index[0]
best_model = trained[best_name]

with open("model/model.pkl", "wb") as f:
    pickle.dump(best_model, f)
with open("model/scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
```

Milestone 4: Flask Backend — Implementation Detail

Activity 4.1 — Loading the model and defining routes:

```
model = pickle.load(open("model/model.pkl", "rb"))
scaler = pickle.load(open("model/scaler.pkl", "rb"))

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/about")
def about():
    return render_template("about.html")
```

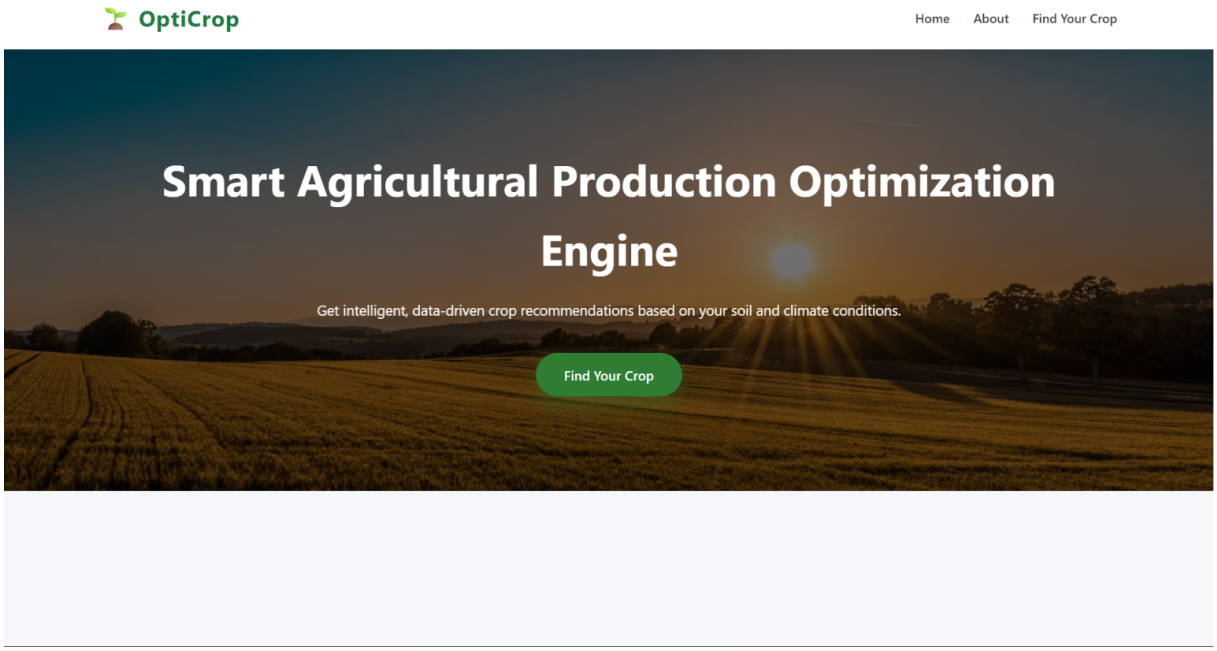
Activity 4.2 — Handling the prediction request:

```
@app.route("/findyourcrop", methods=["GET", "POST"])
def find_your_crop():
    prediction = None
    if request.method == "POST":
        try:
            N = float(request.form["nitrogen"])
            P = float(request.form["phosphorous"])
            K = float(request.form["potassium"])
            temperature = float(request.form["temperature"])
            humidity = float(request.form["humidity"])
            ph = float(request.form["ph"])
            rainfall = float(request.form["rainfall"])

            input_data = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
            scaled_data = scaler.transform(input_data)
            result = model.predict(scaled_data)[0]
            prediction = result.capitalize()
        except Exception as e:
            prediction = f"Error: {e}"
    return render_template("find_your_crop.html", prediction=prediction)
```

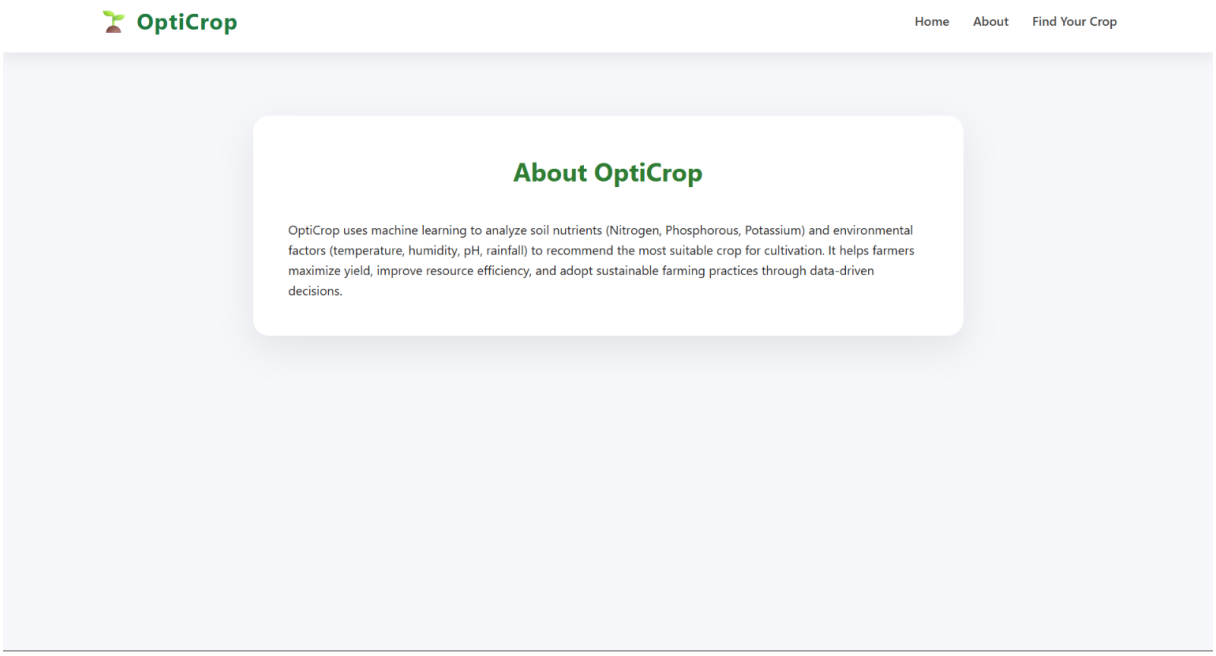
Exploring the Website's Web Pages:

Home Page:



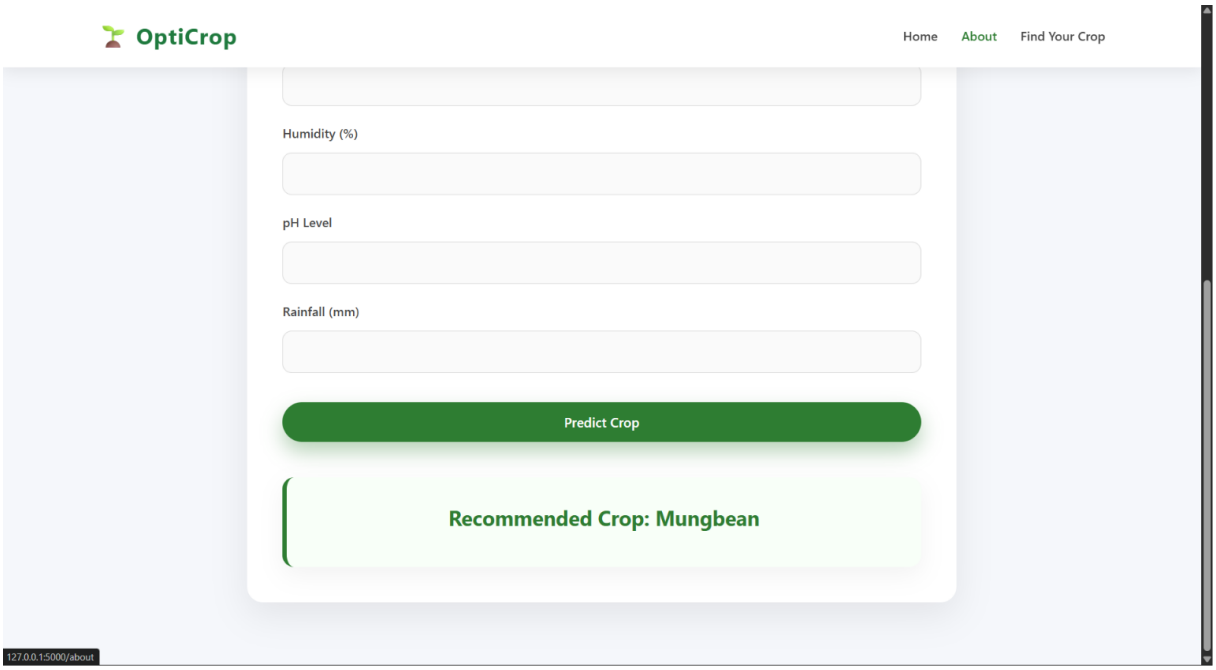
Description: The landing page introduces OptiCrop with a hero banner, the headline "Smart Agricultural Production Optimization Engine," a short description of the tool, a top navigation bar (Home, About, Find Your Crop), and a "Find Your Crop" call-to-action button.

About Page:



Description: Explains how OptiCrop uses machine learning on soil nutrients (Nitrogen, Phosphorous, Potassium) and environmental factors (temperature, humidity, pH, rainfall) to recommend suitable crops, improve yield, resource efficiency, and support sustainable farming through data-driven decisions.

Find Your Crop Page (Input & Result):



Description: A form collecting Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall values, with a "Predict Crop" button. After submission, the recommended crop (e.g., "Recommended Crop: Mungbean") is displayed instantly in a highlighted result box directly below the form.

Conclusion:

OptiCrop is a machine learning platform built with Flask that streamlines crop selection for farmers, researchers, and policymakers. It compares KNN, Logistic Regression, Decision Tree, and Random Forest on soil and environmental data to instantly generate tailored crop recommendations from seven input parameters. The project — spanning data collection, EDA, preprocessing, model comparison, and Flask deployment — highlights how a structured ML pipeline can turn raw agricultural data into practical, farmer-facing decisions, with clear potential for future scalability. Looking ahead, OptiCrop offers opportunities for expansion such as integrating a live weather API, adding a researcher/policymaker dashboard for regional trend analysis, and deploying to cloud hosting with a production-grade server. By continuously refining the model with real-world data and extending the interface, the platform is well-positioned to grow from a demonstration tool into a genuinely useful aid for sustainable farming decisions.